

Contents

1 PLATFORM CONSTITUTION V2 — ServiceManager.AI	6
2 1. Цель платформы	6
3 2. Главные архитектурные инварианты	6
3.1 2.1 Multi-tenant инвариант	7
3.2 2.2 Backend — источник истины	7
3.3 2.3 Service-layer правило	7
4 3. Capability vs Data Scope	7
4.1 3.1 Capability (Permission)	7
4.2 3.2 Data Scope (Policy)	8
5 4. Архитектурные слои	8
5.1 4.1 Controller	8
5.2 4.2 Guard	9
5.3 4.3 Policy	9
5.4 4.4 Service	9
6 5. Permission-Based Access Control (PBAC)	9
6.1 5.1 PermissionBlock	9
6.2 5.2 RolePermission	10
7 6. Domain Event Architecture	10
8 7. Ticket System	10
9 8. Ticket Assignment	10
10 9. Claim механизм	11
11 10. Ticket Board	11
12 11. SLA foundation	11
13 12. Запрещённые архитектурные практики	12
14 13. Долгосрочная архитектура	12
15 14. Главная цель архитектуры	12
16 ARCHITECTURE — ServiceManager.AI	12
17 1. Архитектурные принципы	13
17.1.1 Multi-tenant архитектура	13
17.2.1.2 API-first	13
17.3.1.3 Модульная архитектура (NestJS)	13
18 2. Слои архитектуры	14
18.1.2.1 Controller	14
18.2.2.2 Guards	14
18.3.2.3 Policy Layer	14
18.4.2.4 Service Layer	15
18.5.2.5 Domain Events	15
19 3. Модель доступа	15
19.1.3.1 PermissionBlock	16

19.23.2 RolePermission	16
19.33.3 UserPermission (будущее)	16
204. Основные сущности системы	16
20.1 Company	16
20.2 User	17
20.3 Specialization	17
20.4 ProblemCategory	17
20.5 Ticket	17
215. Назначение тикетов	18
226. Claim механизм	18
237. Ticket Board	18
248. SLA foundation	19
259. Domain Event Store	19
2610. Долгосрочная архитектура	19
2711. Архитектурные ограничения	20
2812. Архитектурная цель	20
29MODULE MAP — ServiceManager.AI	20
301. Общая структура проекта	20
312. Auth Module	21
323. Users Module	21
334. Company Module	22
345. Tickets Module	22
356. Specializations Module	22
367. Problem Categories Module	23
378. Technicians Module	23
389. Permissions Module	23
3910. Events Module	24
4011. Prisma Module	24
4112. Common Module	24
4213. Основные зависимости модулей	25
4314. Центральные модули системы	25
4415. Принцип развития модулей	25
4516. Архитектурная цель	26
46SERVICE BOUNDARIES — ServiceManager.AI	26

471. Архитектурный подход	26
482. Основные доменные зоны	26
493. Identity Domain	26
504. Company Domain	27
515. Ticket Domain	27
526. Workforce Domain	28
537. Problem Domain	28
548. Permission Domain	28
559. Event Domain	29
5610. Analytics Domain (future)	29
5711. SLA Domain (future)	29
5812. Billing Domain (future)	29
5913. Domain dependencies	29
6014. Что запрещено	30
6115. Граница модулей	30
6216. Архитектурная цель	30
63DB SCHEMA EXPLAINED — ServiceManager.AI	30
641. Общая модель данных	31
652. Основные таблицы	31
663. Company	31
674. User	32
685. Ticket	32
696. Specialization	33
707. ProblemCategory	33
718. TechnicianSpecialization	33
729. ProblemCategorySpecialization	34
7310. PermissionBlock	34
7411. RolePermission	34
7512. DomainEvent	35
7613. Parent / Child Tickets	35
7714. Multi-tenant правило	35

7815. Автоназначение	36
7916. Будущие таблицы	36
8017. Важные правила миграций	36
8118. Архитектурная цель базы данных	36
82 SECURITY MODEL — ServiceManager.AI	37
831. Multi-tenant Isolation	37
842. JWT Authentication	37
84.12.1 JWT Payload	38
84.22.2 JWT Secret	38
84.32.3 Token lifetime	38
853. Guards	38
85.13.1 JwtAuthGuard	38
85.23.2 RolesGuard	39
85.33.3 PermissionGuard	39
864. Permission-Based Access Control (PBAC)	39
86.14.1 PermissionBlock	39
86.24.2 RolePermission	40
86.34.3 UserPermission (future)	40
875. Policy Layer (Data Scope)	40
87.15.1 Scope уровни	40
87.25.2 Пример	41
886. Ticket Access Rules	41
897. Domain Event Audit	41
908. Secrets Management	42
919. API Security	42
9210. Будущие улучшения безопасности	42
9311. Критические инварианты безопасности	42
9412. Security Philosophy	43
95 SCALING STRATEGY — ServiceManager.AI	47
961. Текущий уровень архитектуры	47
972. Принцип масштабирования	47
983. Stage 1 — Modular Monolith	47
994. Stage 2 — Optimized Monolith	48
99.0.14.1 Redis cache	48
99.0.24.2 Query optimization	48
99.0.34.3 Background jobs	48
106. Stage 3 — Distributed Services	48

106. Stage 4 — Platform Architecture	49
107. Event driven architecture	49
108. Scaling database	49
109. Scaling API	50
1010. File storage	50
1011. Caching strategy	50
1012. Queue architecture	50
1013. Monitoring	51
1014. Logging	51
1105. Архитектурная цель	51
11ARCHITECTURE DECISIONS — ServiceManager.AI	51
11ADR-001 — Multi-tenant architecture	51
112.Контекст	51
112.Решение	52
112.Последствия	52
11ADR-002 — Backend API-first architecture	52
113.Контекст	52
113.Решение	52
113.Последствия	52
11ADR-003 — Service Layer principle	52
114.Контекст	53
114.Решение	53
114.Последствия	53
11ADR-004 — Permission Based Access Control	53
115.Контекст	53
115.Решение	53
115.Последствия	53
11ADR-005 — Capability vs Data Scope	54
116.Контекст	54
116.Решение	54
11ADR-006 — Domain Event Store	54
117.Контекст	54
117.Решение	54
117.Последствия	54
11ADR-007 — Ticket Board architecture	55
118.Контекст	55
118.Решение	55
118.Последствия	55
11ADR-008 — Auto assignment	55
119.Контекст	55
119.Решение	55

12ADR-009 — Claim mechanism	55
120.Контекст	56
120.Решение	56
12ADR-010 — Docker development environment	56
121.Контекст	56
121.Решение	56
12ADR-011 — Prisma ORM	56
122.Контекст	56
122.Решение	56
12ADR-012 — Architecture documentation	56
123.Контекст	57
123.Решение	57
12Добавление новых ADR	57

1 PLATFORM CONSTITUTION V2 — ServiceManager.AI

Статус: Active

Тип: Архитектурная конституция платформы

Этот документ фиксирует фундаментальные правила архитектуры ServiceManager.AI.

Нарушение этих правил приводит к архитектурной деградации системы.

Документ обязателен для:

- разработчиков
- AI-ассистентов
- архитекторов системы

2 1. Цель платформы

ServiceManager.AI — это SaaS-платформа управления сервисной сетью.

Платформа должна масштабироваться до:

- крупных сервисных компаний
- сетевых структур
- франчайзинговых сетей
- enterprise-клиентов

Система должна быть:

- multi-tenant
- безопасной
- масштабируемой
- расширяемой

3 2. Главные архитектурные инварианты

Инварианты — это правила, которые **никогда нельзя нарушать**.

3.1 2.1 Multi-tenant инвариант

Каждая доменная сущность обязана содержать:

companyId

Все запросы к базе должны фильтроваться по:

req.user.companyId

Запрещено:

- глобальные выборки
 - cross-tenant доступ
 - отсутствие фильтра companyId
-

3.2 2.2 Backend — источник истины

Backend является единственным источником бизнес-логики.

Запрещено:

- переносить бизнес-логику во frontend
 - дублировать бизнес-логику
-

3.3 2.3 Service-layer правило

Бизнес-логика размещается только в:

Service

Controller отвечает только за:

- HTTP
 - DTO
 - передачу данных
-

4 3. Capability vs Data Scope

Архитектура доступа разделена на два уровня.

Capability

↓

Data Scope

4.1 3.1 Capability (Permission)

Capability отвечает на вопрос:

МОЖНО ЛИ ВЫПОЛНИТЬ ДЕЙСТВИЕ

Пример:

TICKETS_ASSIGN

TICKETS_VIEW

TICKETS_CLAIM
TICKETS_STATUS_CHANGE
Capability проверяется через:
PermissionGuard

4.2 3.2 Data Scope (Policy)

Scope отвечает на вопрос:
К КАКИМ ДАННЫМ ПРИМЕНИМО ДЕЙСТВИЕ
Примеры scope:
ALL
COMPANY
ASSIGNED_TO_ME
CREATED_BY_ME
Scope реализуется через:
RoleScopePolicy

5 4. Архитектурные слои

Система использует слоистую архитектуру.

Controller
↓
Guard (PermissionGuard)
↓
Policy Layer
↓
Service
↓
Domain Events
↓
Database (Prisma)

5.1 4.1 Controller

Отвечает за:

- HTTP маршруты
- DTO
- вызов service

Controller не содержит бизнес-логики.

5.2 4.2 Guard

Guard проверяет:

можно ли выполнить действие.

5.3 4.3 Policy

Policy определяет:

какие данные доступны.

5.4 4.4 Service

Service выполняет:

- бизнес-операции
 - работу с базой
 - создание Domain Events
-

6 5. Permission-Based Access Control (PBAC)

Система использует PBAC модель.

Модель:

Role

+

Permission Blocks

6.1 5.1 PermissionBlock

Таблица:

PermissionBlock

Поля:

id

code

name

description

Примеры permission:

TICKETS_CREATE

TICKETS_ASSIGN

TICKETS_VIEW

TICKETS_CLAIM

TICKETS_STATUS_CHANGE

USERS_MANAGE

ANALYTICS_VIEW

6.2 5.2 RolePermission

Связь:

role → permissionBlock

Это позволяет:

- гибко управлять правами
 - расширять систему
 - создавать feature-packages
-

7 6. Domain Event Architecture

Любое важное действие фиксируется в Event Store.

Таблица:

DomainEvent

Примеры событий:

ticket.created
ticket.assigned
ticket.claimed
ticket.status_changed

Event Store используется для:

- аудита
 - аналитики
 - SLA
 - workflow engine
-

8 7. Ticket System

Ticket — центральная сущность платформы.

Ticket содержит:

id
companyId
problemCategoryId
problemText
urgency
status
assignedTechnicianId
slaMinutes

9 8. Ticket Assignment

Алгоритм назначения:

1. Определить специализации категории проблемы
2. Найти техников с этими специализациями

3. Если autoAssignEnabled:
назначить первого кандидата
4. Иначе:
оставить NEW

10 9. Claim механизм

Техник может забирать заявки.
API:
GET /tickets/available
POST /tickets/:id/claim
После claim:
assignedTechnicianId обновляется.

11 10. Ticket Board

Основной интерфейс системы.
Колонки:
NEW
ASSIGNED
IN_PROGRESS
DONE
CANCELED
API:
GET /tickets/board
Board поддерживает фильтры:
status
assigneeId
sla
search

12 11. SLA foundation

Каждый тикет содержит:
slaMinutes
Будущий SLA engine будет вычислять:
deadline
atRisk
breached

13 12. Запрещённые архитектурные практики

Нельзя:

- писать бизнес-логику в controller
 - обходить guards
 - писать raw SQL без причины
 - делать запросы без companyId
 - хардкодить токены
-

14 13. Долгосрочная архитектура

Платформа развивается к:

Service Network Platform.

Будущие модули:

SLA Engine
Zones / Territories
Workflow Engine
Analytics Engine
Billing
Files / Media
Comments
Mobile API

15 14. Главная цель архитектуры

ServiceManager.AI должен стать enterprise-уровня платформой управления сервисной сетью.

Ключевые элементы:

- PBAC
- Policy Layer
- Event Store
- SLA Engine
- Analytics

16 ARCHITECTURE — ServiceManager.AI

Этот документ описывает текущую архитектуру платформы.

ServiceManager.AI — это SaaS-платформа управления сервисными заявками для сервисных компаний и сетей.

Технологический стек:

- Backend: NestJS
 - ORM: Prisma
 - Database: PostgreSQL
 - Auth: JWT
 - Architecture: Modular + PBAC
 - Deployment: Docker
-

17 1. Архитектурные принципы

17.1 1.1 Multi-tenant архитектура

Каждая доменная сущность содержит:

`companyId`

Это обеспечивает:

- изоляцию данных между компаниями
- возможность SaaS-масштабирования
- поддержку enterprise-клиентов

Любой запрос обязан фильтроваться по `companyId`.

Источник `companyId`:

JWT payload → `req.user.companyId`

Запрещено:

- доступ к данным другой компании
 - глобальные выборки без `companyId`
-

17.2 1.2 API-first

Backend является API-платформой.

Frontend, mobile app и интеграции работают через API.

Все изменения должны учитывать:

- обратную совместимость API
 - стабильность контрактов
 - масштабируемость
-

17.3 1.3 Модульная архитектура (NestJS)

Система разделена на модули:

`AuthModule`

`UsersModule`

`CompanyModule`

`TicketsModule`

`SpecializationsModule`

`ProblemCategoriesModule`

`TechniciansModule`

`PermissionsModule`

`EventsModule`

Каждый модуль содержит:

`controller`

`service`

`dto`

`module`

Бизнес-логика размещается **только в service**.

Контроллеры не содержат бизнес-логики.

18 2. Слои архитектуры

Платформа использует слоистую модель:

Controller

↓

Guard / PermissionGuard

↓

Policy Layer (RoleScopePolicy)

↓

Service Layer

↓

Domain Events

↓

Prisma / Database

18.1 2.1 Controller

Controller отвечает только за:

- HTTP маршруты
- валидацию DTO
- передачу данных в service

Controller не должен:

- выполнять бизнес-логику
 - работать с Prisma напрямую
-

18.2 2.2 Guards

Guards проверяют **можно ли выполнять действие**.

Примеры:

RolesGuard

PermissionGuard

Guard проверяет:

- JWT
- роль пользователя
- permission block

Guard **не фильтрует данные**.

18.3 2.3 Policy Layer

Policy отвечает за:

какие данные пользователь может видеть.

Пример:

RoleScopePolicy

Scope:

ALL

COMPANY

ASSIGNED_TO_ME

CREATED_BY_ME

Policy применяется на уровне service.

18.4 2.4 Service Layer

Service содержит:

- бизнес-логику
- правила системы
- работу с Prisma
- вызов Domain Events

Service — главный слой бизнес-логики.

18.5 2.5 Domain Events

Любые важные изменения записываются в Event Store.

Таблица:

DomainEvent

Пример событий:

ticket.created

ticket.assigned

ticket.claimed

ticket.status_changed

Это используется для:

- аудита
 - аналитики
 - SLA
 - будущего event-driven архитектурного перехода
-

19 3. Модель доступа

Система использует **PBAC (Permission-Based Access Control)**.

Модель:

Role

+

PermissionBlocks

19.1 3.1 PermissionBlock

Таблица:

PermissionBlock

Поля:

id
code
name
description

Примеры:

TICKETS_VIEW
TICKETS_VIEW_AVAILABLE
TICKETS_ASSIGN
TICKETS_CLAIM
TICKETS_STATUS_CHANGE
USERS_MANAGE
COMPANY_SETTINGS_EDIT
ANALYTICS_VIEW

19.2 3.2 RolePermission

Связь:

role → permissionBlock

Позволяет:

- гибко настраивать роли
 - добавлять новые функции без изменения кода
 - делать feature-packages
-

19.3 3.3 UserPermission (будущее)

Позволяет:

давать индивидуальные permission пользователям поверх роли.

Это нужно для enterprise-клиентов.

20 4. Основные сущности системы

20.1 Company

Компания внутри SaaS.

Поля:

id
name
autoAssignEnabled

20.2 User

Пользователь системы.

Поля:

id
companyId
email
password
role

20.3 Specialization

Специализация техника.

Примеры:

электрика
IT
холодильники

20.4 ProblemCategory

Категория проблемы.

Связана со специализациями.

Используется для:

- автоназначения
 - аналитики
 - инструкций
-

20.5 Ticket

Основная сущность системы.

Поля:

id
companyId
parentId
requesterName
requesterPhone
address
pointName
problemCategoryId
problemText
urgency
status
slaMinutes
assignedTechnicianId

21 5. Назначение тикетов

Алгоритм назначения:

- 1) определить специализации категории проблемы
 - 2) найти техников с этими специализациями
 - 3) если `autoAssignEnabled = true`
→ назначить первого кандидата
 - 4) иначе
→ тикет остаётся NEW
- и возвращается список кандидатов.
-

22 6. Claim механизм

Техник может забирать NEW заявки.

API:

GET /tickets/available
POST /tickets/:id/claim

Техник видит:

NEW заявки своей специализации.

После claim:

`assignedTechnicianId = technicianId`

23 7. Ticket Board

Board — основной интерфейс работы.

Колонки:

NEW
ASSIGNED
IN_PROGRESS
DONE
CANCELED

API:

GET /tickets/board

Board возвращает:

columns
cards
meta

Фильтры:

status
assigneeId

sla
search

24 8. SLA foundation

Каждый тикет содержит:

slaMinutes

Будущий SLA-движок будет считать:

deadline

atRisk

breached

Board API уже учитывает SLA-фильтры.

25 9. Domain Event Store

Все критические действия записываются в:

DomainEvent

Поля:

id

companyId

entityType

entityId

type

payload

createdAt

Это база для:

audit log

analytics

SLA engine

workflow engine

26 10. Долгосрочная архитектура

Платформа развивается к:

Service Network Platform.

Будущие модули:

SLA Engine

Zones / Territories

Workflow Engine

Analytics Engine

Billing

Files / Media

27 11. Архитектурные ограничения

Запрещено:

- писать бизнес-логику в controller
 - делать запросы без companyId
 - использовать raw SQL без причины
 - обходить permission checks
 - хардкодить токены
-

28 12. Архитектурная цель

ServiceManager.AI должен стать:

enterprise-уровня платформой управления сервисной сетью.

Ключевые элементы:

- PBAC
- Policy Layer
- Event Store
- SLA Engine
- Analytics

Это переход от MVP к платформенной архитектуре.

29 MODULE MAP — ServiceManager.AI

Этот документ описывает структуру backend модулей системы.

Backend построен на:

NestJS modular architecture.

Каждый модуль содержит:

controller
service
dto
module

30 1. Общая структура проекта

backend/
src/
auth/
users/
company/

tickets/
specializations/
problem-categories/
technicians/
permissions/
events/
prisma/
common/

31 2. Auth Module

Путь:

src/auth

Файлы:

auth.controller.ts
auth.service.ts
auth.module.ts

dto/

login.dto.ts
register.dto.ts

Назначение:

- регистрация компаний
- логин пользователей
- выдача JWT

API:

POST /auth/register
POST /auth/login
GET /auth/me

32 3. Users Module

Путь:

src/users

Назначение:

управление пользователями компании.

Основные роли:

ADMIN
MASTER
TECHNICIAN
CLIENT
NETWORK_DIRECTOR
STAFF

Основные операции:

- создание пользователей
 - список пользователей
 - управление ролями
-

33 4. Company Module

Путь:

src/company

Назначение:

настройки компании.

Основные функции:

- получение информации о компании
- включение автоназначения

API:

GET /company

PATCH /company/auto-assign

34 5. Tickets Module

Путь:

src/tickets

Это центральный модуль системы.

Основные функции:

- создание тикетов
- назначение техников
- claim механизм
- изменение статусов
- ticket board

Основные API:

POST /tickets

GET /tickets

GET /tickets/:id

PUT /tickets/:id/assign/:technicianId

POST /tickets/:id/claim

PATCH /tickets/:id/status

GET /tickets/board

35 6. Specializations Module

Путь:

src/specializations

Назначение:

управление специализациями.

Примеры:

- электрика
 - IT
 - холодильное оборудование
-

36 7. Problem Categories Module

Путь:

src/problem-categories

Назначение:

категории проблем.

Категория содержит:

- название
- инструкции
- связанные специализации

Используется для:

- автоназначения
 - аналитики
-

37 8. Technicians Module

Путь:

src/technicians

Назначение:

управление техниками.

Функции:

- список техников
- привязка специализаций
- поиск кандидатов

Используется в:

Ticket assignment.

38 9. Permissions Module

Путь:

src/permissions

Назначение:

РВАС система.

Основные сущности:

PermissionBlock

RolePermission

Функции:

- управление permission блоками
- проверка доступов

Guard:

PermissionGuard

39 10. Events Module

Путь:

src/events

Назначение:

Domain Event Store.

Любое важное действие записывается в таблицу:

DomainEvent

Примеры событий:

ticket.created

ticket.assigned

ticket.claimed

ticket.status_changed

40 11. Prisma Module

Путь:

src/prisma

Назначение:

работа с базой данных.

Файл:

prisma.service.ts

Используется всеми сервисами.

41 12. Common Module

Путь:

src/common

Содержит:

guards
decorators
constants
utils

Примеры:

PermissionGuard
permission.constants.ts

42 13. Основные зависимости модулей

Auth → Users

Users → Company

Tickets → Users

Tickets → Technicians

Tickets → ProblemCategories

Tickets → Specializations

Tickets → Events

Tickets → Permissions

Technicians → Specializations

ProblemCategories → Specializations

43 14. Центральные модули системы

Ключевые модули:

Tickets

Permissions

Events

Они формируют ядро платформы.

44 15. Принцип развития модулей

Новый модуль должен:

1. иметь собственный сервис
2. иметь DTO
3. быть подключен в AppModule
4. соблюдать multi-tenant архитектуру

Запрещено:

- смешивать модули
 - писать бизнес-логику в controller
-

45 16. Архитектурная цель

Модули должны оставаться:

- изолированными
- расширяемыми
- независимыми

Это позволяет системе масштабироваться как enterprise SaaS платформа.

46 SERVICE BOUNDARIES — ServiceManager.AI

Этот документ описывает границы сервисов системы.

Цель документа:

- определить доменные зоны системы
 - предотвратить смешивание ответственности модулей
 - упростить масштабирование архитектуры
-

47 1. Архитектурный подход

ServiceManager.AI построен по принципам:

Domain Driven Design (DDD)

Система разделена на:

Bounded Contexts.

Каждый контекст отвечает за свою область логики.

48 2. Основные доменные зоны

Платформа разделена на следующие зоны:

Identity Domain
Company Domain
Ticket Domain
Workforce Domain
Permission Domain
Event Domain
Analytics Domain

49 3. Identity Domain

Отвечает за:

аутентификацию пользователей.

Модуль:

auth

Функции:

- регистрация
- логин
- выдача JWT
- идентификация пользователя

API:

POST /auth/register

POST /auth/login

GET /auth/me

50 4. Company Domain

Отвечает за:

данные компании.

Модуль:

company

Функции:

- настройки компании
- автоназначение
- конфигурация системы

API:

GET /company

PATCH /company/auto-assign

51 5. Ticket Domain

Центральный домен системы.

Модуль:

tickets

Функции:

- создание тикетов
- управление статусами
- назначение техников
- claim механизм
- board

Основные API:

POST /tickets

GET /tickets

GET /tickets/:id

POST /tickets/:id/claim

PUT /tickets/:id/assign/:technicianId

PATCH /tickets/:id/status

GET /tickets/board

52 6. Workforce Domain

Отвечает за техников и компетенции.

Модули:

users
technicians
specializations

Функции:

- управление пользователями
- управление техниками
- управление специализациями

Связи:

Technician → Specialization

53 7. Problem Domain

Отвечает за классификацию проблем.

Модуль:

problem-categories

Функции:

- категории проблем
- инструкции
- связь с специализациями

Используется в:

ticket assignment

54 8. Permission Domain

Отвечает за систему прав.

Модуль:

permissions

Функции:

- permission blocks
- role permissions
- проверка доступов

Основной guard:

PermissionGuard

55 9. Event Domain

Отвечает за Event Store.

Модуль:

events

Функции:

- запись событий
- audit log
- источник аналитики

Таблица:

DomainEvent

56 10. Analytics Domain (future)

Будущий домен.

Будет отвечать за:

- метрики
 - dashboards
 - SLA анализ
-

57 11. SLA Domain (future)

Будущий домен.

Будет отвечать за:

- SLA policies
 - deadline calculation
 - breach detection
-

58 12. Billing Domain (future)

Будущий домен.

Будет отвечать за:

- тарифы
 - подписки
 - биллинг
-

59 13. Domain dependencies

Зависимости доменов:

Ticket Domain
→ Workforce Domain
→ Problem Domain
→ Permission Domain
→ Event Domain

Workforce Domain
→ Company Domain

Permission Domain
→ Identity Domain

60 14. Что запрещено

Нельзя:

- смешивать домены
- делать cross-module логику
- обращаться напрямую к чужим таблицам

Взаимодействие должно происходить через:
service layer.

61 15. Граница модулей

Каждый модуль должен:

- иметь свой service
- иметь свой controller
- иметь DTO

Модули должны быть:
изолированными.

62 16. Архитектурная цель

ServiceManager.AI должен развиваться как модульная SaaS платформа.

Это позволит в будущем:

- разделять сервисы
- масштабировать систему
- внедрять новые домены

63 DB SCHEMA EXPLAINED — ServiceManager.AI

Этот документ объясняет структуру базы данных системы.

Он дополняет файл:

prisma/schema.prisma

Цель документа:

- объяснить назначение таблиц
 - описать связи
 - предотвратить ошибки при миграциях
-

64 1. Общая модель данных

ServiceManager.AI — multi-tenant SaaS система.

Все доменные сущности содержат:

companyId

Это обеспечивает:

- изоляцию данных
 - масштабируемость
-

65 2. Основные таблицы

Основные таблицы системы:

Company

User

Ticket

Specialization

ProblemCategory

TechnicianSpecialization

ProblemCategorySpecialization

PermissionBlock

RolePermission

DomainEvent

66 3. Company

Компания внутри SaaS.

Таблица:

Company

Основные поля:

id

name

autoAssignEnabled

Назначение:

- изоляция данных
- настройки компании

Связи:

Company → Users

Company → Tickets

Company → Specializations
Company → ProblemCategories

67 4. User

Пользователь системы.

Таблица:

User

Основные поля:

id
companyId
email
password
role

Роли:

ADMIN
MASTER
TECHNICIAN
CLIENT
NETWORK_DIRECTOR
STAFF

Связи:

User → Company
User → Tickets (assignedTechnicianId)

68 5. Ticket

Основная сущность системы.

Таблица:

Ticket

Основные поля:

id
companyId
parentId
problemCategoryId
problemText
urgency
status
assignedTechnicianId
slaMinutes

Назначение:

управление сервисными заявками.

Связи:

Ticket → Company
Ticket → ProblemCategory
Ticket → User (technician)
Ticket → Ticket (parent-child)

69 6. Specialization

Специализация техника.

Таблица:

Specialization

Примеры:

электрика

IT

кофемашины

холодильное оборудование

Связи:

Specialization → TechnicianSpecialization

Specialization → ProblemCategorySpecialization

70 7. ProblemCategory

Категория проблемы.

Таблица:

ProblemCategory

Основные поля:

id

companyId

name

instructions

Категория содержит инструкции и связана со специализациями.

Связи:

ProblemCategory → Ticket

ProblemCategory → ProblemCategorySpecialization

71 8. TechnicianSpecialization

Связующая таблица.

Связывает:

User (technician)

Specialization

Тип связи:

many-to-many

Назначение:

определяет компетенции техников.

72 9. ProblemCategorySpecialization

Связующая таблица.

Связывает:

ProblemCategory
Specialization

Тип связи:

many-to-many

Назначение:

определяет какие специализации подходят для категории проблемы.

73 10. PermissionBlock

Таблица permission системы.

Поля:

id
code
name
description

Примеры:

TICKETS_VIEW
TICKETS_ASSIGN
TICKETS_CLAIM
TICKETS_STATUS_CHANGE
USERS_MANAGE

74 11. RolePermission

Связь:

role → permissionBlock

Позволяет:

гибко настраивать права ролей.

75 12. DomainEvent

Event store системы.

Таблица:

DomainEvent

Поля:

id
companyId
entityType
entityId
type
payload
createdAt

Примеры событий:

ticket.created
ticket.assigned
ticket.claimed
ticket.status_changed

Назначение:

- audit log
 - аналитика
 - SLA engine
-

76 13. Parent / Child Tickets

Ticket поддерживает:

иерархию заявок.

Поле:

parentId

Это позволяет:

- создавать дочерние задачи
 - разбивать сложные заявки
-

77 14. Multi-tenant правило

Все таблицы доменной модели содержат:

companyId

Любой запрос обязан фильтроваться по:

companyId

Нарушение этого правила ломает безопасность системы.

78 15. Автоназначение

Алгоритм использует таблицы:

ProblemCategory
ProblemCategorySpecialization
TechnicianSpecialization

Процесс:

1. определить специализации категории
 2. найти техников с этими специализациями
 3. выбрать кандидатов
-

79 16. Будущие таблицы

Планируется добавить:

SLA Policy
Zone
Territory
Point
Workflow
Attachment
Comment
BillingPlan
Subscription

80 17. Важные правила миграций

При изменении schema.prisma:

нельзя:

- удалять поля без анализа
- ломать связи
- менять типы без миграции

AI обязан:

- объяснить влияние изменений
 - предложить имя миграции
-

81 18. Архитектурная цель базы данных

База должна поддерживать:

- multi-tenant SaaS
- аналитические запросы
- SLA engine
- workflow engine
- масштабирование

82 SECURITY MODEL — ServiceManager.AI

Этот документ описывает модель безопасности платформы.

ServiceManager.AI — мульти-тенант SaaS система, поэтому безопасность строится вокруг строгой изоляции компаний.

Основные принципы:

- multi-tenant isolation
 - JWT authentication
 - PBAC access control
 - policy-based data visibility
 - audit via domain events
-

83 1. Multi-tenant Isolation

Каждая доменная сущность обязана содержать:

`companyId`

Примеры:

`User`

`Ticket`

`Specialization`

`ProblemCategory`

`DomainEvent`

Любой запрос к базе обязан включать:

`companyId = req.user.companyId`

Источник `companyId`:

JWT payload.

Пример:

`req.user.companyId`

Запрещено:

- выполнять запросы без `companyId`
- возвращать данные других компаний
- делать глобальные выборки

Это критический инвариант SaaS.

84 2. JWT Authentication

Аутентификация строится на JWT.

Token создаётся через:

POST `/auth/login`

или

POST `/auth/register`

84.1 2.1 JWT Payload

JWT содержит:

```
{ sub: userId, email: string, companyId: string, role: string }
```

Используется для:

- идентификации пользователя
 - multi-tenant фильтрации
 - базовых role checks
-

84.2 2.2 JWT Secret

JWT подписывается:

JWT_SECRET

Хранится только в:

.env

docker-compose env

production secret manager

Запрещено:

- хардкодить JWT_SECRET
 - хранить секреты в коде
 - коммитить .env
-

84.3 2.3 Token lifetime

Текущая модель:

short-lived access token.

Будущее развитие:

refresh tokens

session management

85 3. Guards

Guards отвечают за проверку доступа к действиям.

Примеры:

JwtAuthGuard

RolesGuard

PermissionGuard

85.1 3.1 JwtAuthGuard

Проверяет:

- наличие токена

- подпись JWT
- срок действия

Добавляет в request:

req.user

85.2 3.2 RolesGuard

Проверяет:

role пользователя.

Используется для:

- legacy RBAC
 - совместимости
-

85.3 3.3 PermissionGuard

Основной механизм доступа.

Проверяет наличие:

PermissionBlock

например:

TICKETS_VIEW
TICKETS_VIEW_AVAILABLE
TICKETS_ASSIGN
TICKETS_CLAIM
TICKETS_STATUS_CHANGE

86 4. Permission-Based Access Control (PBAC)

Модель:

Role

+

PermissionBlocks

86.1 4.1 PermissionBlock

Таблица:

PermissionBlock

Поля:

id
code
name
description

Примеры:

TICKETS_VIEW
TICKETS_VIEW_AVAILABLE
TICKETS_ASSIGN
TICKETS_CLAIM
TICKETS_STATUS_CHANGE
USERS_MANAGE
COMPANY_SETTINGS_EDIT
ANALYTICS_VIEW

86.2 4.2 RolePermission

Связь:

role → permissionBlock

Это позволяет:

- менять права ролей без изменения кода
 - добавлять новые функции
 - продавать feature пакеты
-

86.3 4.3 UserPermission (future)

Позволяет:

назначать permission напрямую пользователю.

Используется для:

enterprise кастомизаций.

87 5. Policy Layer (Data Scope)

Permissions отвечают на вопрос:

можно ли действие

Policy отвечает на вопрос:

к каким данным применимо действие

87.1 5.1 Scope уровни

ALL
COMPANY
ASSIGNED_TO_ME
CREATED_BY_ME

87.2 5.2 Пример

TECHNICIAN:

может:

TICKETS_VIEW

но score:

ASSIGNED_TO_ME

88 6. Ticket Access Rules

ADMIN

может:

- видеть все тикеты компании
- назначать техников
- менять статусы
- управлять настройками

MASTER / DISPATCHER

может:

- создавать тикеты
- назначать техников
- управлять заявками

TECHNICIAN

может:

- видеть назначенные тикеты
- видеть доступные NEW заявки
- делать claim
- менять статус

CLIENT

может:

- создавать заявки
 - видеть свои заявки
-

89 7. Domain Event Audit

Любые критические действия фиксируются.

Таблица:

DomainEvent

Примеры:

ticket.created

ticket.assigned

ticket.claimed

ticket.status_changed

Это обеспечивает:

- audit trail
 - расследование инцидентов
 - аналитику
-

90 8. Secrets Management

Секреты могут храниться только в:

.env
docker-compose
secret manager (production)

Запрещено:

- хардкодить секреты
 - хранить секреты в Git
 - публиковать токены
-

91 9. API Security

Все защищённые endpoints требуют:

Authorization: Bearer

Пример:

curl http://localhost:3000/auth/me
-H "Authorization: Bearer TOKEN"

92 10. Будущие улучшения безопасности

Планируемые улучшения:

refresh tokens
rate limiting
API throttling
IP restrictions
company API keys
security audit log

93 11. Критические инварианты безопасности

Нельзя:

- обходить guards
- отключать companyId фильтрацию
- использовать raw SQL без фильтра
- возвращать cross-company данные

Любое нарушение этих правил ломает multi-tenant безопасность системы.

94 12. Security Philosophy

Безопасность платформы строится вокруг:

strict tenant isolation
least privilege access
auditable actions

Это позволяет системе работать как enterprise SaaS.

```
cat > docs/OBSERVABILITY_STRATEGY.md «'EOF' # OBSERVABILITY STRATEGY — ServiceMan-  
ager.AI
```

Этот документ описывает стратегию наблюдаемости системы (Observability).

Цель:

- понимать состояние системы
- быстро находить ошибки
- отслеживать производительность
- контролировать рост системы

1. Что такое Observability

Observability — это способность системы объяснить своё состояние через:

- логи
- метрики
- трассировку

В SaaS системе это критически важно.

2. Три столпа Observability

Observability состоит из трех элементов:

Logs
Metrics
Tracing

3. Logs (логи)

Логи фиксируют события системы.

Примеры логов:

- ошибки API
- создание тикета
- назначение техника
- изменение статуса

Каждый лог должен содержать:

timestamp
level
message
context

4. Уровни логирования

Используются уровни:

debug

log

warn

error

debug

используется только в разработке

log

обычные события системы

warn

подозрительные ситуации

error

ошибки системы

5. Логирование в NestJS

Использовать:

NestJS Logger

Пример:

```
this.logger.log('Ticket created')
```

Запрещено использовать:

```
console.log
```

6. Структура логов

Каждый лог должен содержать:

timestamp

service

message

context

Пример:

timestamp: 2026-03-04T12:00:00Z

service: tickets

message: ticket created

ticketId: 123

7. Error logging

Все ошибки уровня 500 должны логироваться.

Особенно:

- database errors
 - unexpected exceptions
-

8. Metrics (метрики)

Метрики показывают состояние системы.

Примеры метрик:

requests per second
API latency
error rate
tickets created per hour

9. Основные метрики

ServiceManager.AI должен отслеживать:

API requests
API latency
error rate
tickets created
tickets closed
active technicians

10. SLA метрики

Будущие метрики:

SLA breached
SLA warning
average response time

11. Tracing (трассировка)

Tracing показывает как проходит запрос через систему.

Пример цепочки:

API request
→ service
→ database
→ response

Это помогает находить:

slow queries
bottlenecks

12. Correlation ID

Каждый API запрос должен иметь:

correlationId.

Он передается через весь pipeline.

Это позволяет связывать логи одного запроса.

13. Monitoring stack

Рекомендуемый стек:

Prometheus
Grafana

Prometheus собирает метрики.

Grafana визуализирует данные.

14. Logging stack

Для production рекомендуется:

ELK stack

Elasticsearch

Logstash

Kibana

Это позволяет:

- хранить логи
 - искать ошибки
 - анализировать события
-

15. Alerting

Система должна отправлять алерты при проблемах.

Примеры алертов:

error rate > 5%

API latency > 2s

database errors

16. Health checks

Система должна иметь health endpoints.

Пример:

GET /health

Он должен проверять:

database connection

service availability

17. Архитектурная цель

Observability должна позволять:

- быстро находить ошибки
- контролировать рост системы
- анализировать поведение пользователей

Это обязательный элемент production SaaS системы.

18. Главный принцип

Если система не наблюдаема — она не управляемая. EOF

95 SCALING STRATEGY — ServiceManager.AI

Этот документ описывает стратегию масштабирования платформы.

Цель документа:

- подготовить систему к росту
 - избежать архитектурных тупиков
 - определить этапы масштабирования
-

96 1. Текущий уровень архитектуры

ServiceManager.AI сейчас является:

Modular Monolith.

Архитектура:

NestJS
Prisma
PostgreSQL
Docker

Это оптимальная архитектура для:

MVP
раннего SaaS
быстрой разработки

97 2. Принцип масштабирования

Система должна масштабироваться постепенно.

Этапы роста:

Stage 1 — Modular Monolith
Stage 2 — Optimized Monolith
Stage 3 — Distributed Services
Stage 4 — Platform Architecture

Нельзя преждевременно переходить к микросервисам.

98 3. Stage 1 — Modular Monolith

Текущий этап.

Особенности:

- один backend
- одна база данных
- модули внутри NestJS

Преимущества:

- простая разработка
- высокая скорость

- минимальная сложность
-

99 4. Stage 2 — Optimized Monolith

Когда переходить:

100+ компаний
50k+ тикетов в месяц

Добавляется:

99.0.1 4.1 Redis cache

Используется для:

- board cache
 - session cache
 - rate limiting
-

99.0.2 4.2 Query optimization

Добавляются:

- индексы
 - pagination
 - optimized queries
-

99.0.3 4.3 Background jobs

Добавляется:

job queue

Используется для:

- SLA calculation
- аналитики
- отправки уведомлений

Рекомендуемые технологии:

BullMQ
Redis

100 5. Stage 3 — Distributed Services

Когда переходить:

1000+ компаний
миллионы тикетов

Система начинает разделяться на сервисы.

Основные кандидаты для выделения сервисов:

Ticket Service
Analytics Service
Notification Service
File Service

101 6. Stage 4 — Platform Architecture

Когда переходить:

enterprise масштаб.

Система становится платформой сервисов.

Добавляются:

API Gateway
Event Bus
Service Mesh

102 7. Event driven architecture

DomainEvent уже используется как Event Store.

Будущий шаг:

перевести события в Event Bus.

Технологии:

Kafka
NATS
RabbitMQ

103 8. Scaling database

PostgreSQL может масштабироваться достаточно долго.

Этапы:

Stage 1:

одна база

Stage 2:

read replicas

Stage 3:

sharding

104 9. Scaling API

API масштабируется через:

horizontal scaling.

Backend контейнеры:

backend-1

backend-2

backend-3

Балансировка:

load balancer

105 10. File storage

Файлы не должны храниться в базе данных.

Использовать:

S3 storage

Примеры:

AWS S3

MinIO

106 11. Caching strategy

Кэш используется для:

ticket board

analytics

configuration

Технология:

Redis

107 12. Queue architecture

Очереди используются для:

уведомлений

аналитики

SLA engine

workflow engine

Рекомендуемые технологии:

BullMQ

RabbitMQ

108 13. Monitoring

Для production системы необходимо добавить мониторинг.

Инструменты:

Prometheus

Grafana

109 14. Logging

Централизованные логи.

Инструменты:

ELK stack

110 15. Архитектурная цель

ServiceManager.AI должен масштабироваться до уровня enterprise SaaS.

Архитектура должна поддерживать:

- миллионы заявок
- тысячи компаний
- распределенную систему

111 ARCHITECTURE DECISIONS — ServiceManager.AI

Этот документ фиксирует архитектурные решения проекта.

Каждое решение оформляется как ADR (Architecture Decision Record).

ADR отвечает на вопрос:

ПОЧЕМУ архитектура сделана именно так.

Это предотвращает:

- хаотичные изменения
 - повторные споры
 - архитектурную деградацию
-

112 ADR-001 — Multi-tenant architecture

Статус: Accepted

112.1 Контекст

ServiceManager.AI является SaaS системой.

Система должна обслуживать множество компаний в одной базе данных.

112.2 Решение

Каждая доменная сущность содержит:

companyId

Все запросы фильтруются по:

req.user.companyId

112.3 Последствия

Плюсы:

- изоляция данных
- масштабируемость
- SaaS модель

Минусы:

- необходимо строго соблюдать фильтрацию
-

113 ADR-002 — Backend API-first architecture

Статус: Accepted

113.1 Контекст

Платформа должна поддерживать:

- web frontend
- mobile приложение
- интеграции

113.2 Решение

Backend является API-first системой.

Все клиенты работают через API.

113.3 Последствия

Плюсы:

- гибкость архитектуры
- поддержка мобильных клиентов

Минусы:

- необходимость стабильных API контрактов
-

114 ADR-003 — Service Layer principle

Статус: Accepted

114.1 Контекст

Бизнес-логика не должна смешиваться с HTTP обработкой.

114.2 Решение

Controller отвечает только за:

- HTTP
- DTO
- вызов service

Service содержит:

- бизнес-логику
- работу с Prisma

114.3 Последствия

Плюсы:

- чистая архитектура
 - тестируемость
-

115 ADR-004 — Permission Based Access Control

Статус: Accepted

115.1 Контекст

Role-based модель плохо масштабируется.

Большие системы требуют гибкой модели прав.

115.2 Решение

Использовать PBAC.

Модель:

Role

+

Permission Blocks

Permission проверяется через:

PermissionGuard

115.3 Последствия

Плюсы:

- гибкость прав
- возможность feature packages

Минусы:

- более сложная архитектура
-

116 ADR-005 — Capability vs Data Scope

Статус: Accepted

116.1 Контекст

Обычные permission системы не разделяют:

действие
и
доступ к данным

116.2 Решение

Разделить:

Capability
Data Scope

Capability:

можно ли действие

Score:

к каким данным применимо действие

Score реализуется через:

RoleScopePolicy

117 ADR-006 — Domain Event Store

Статус: Accepted

117.1 Контекст

Необходимо:

- аудит действий
- аналитика
- SLA engine

117.2 Решение

Ввести таблицу:

DomainEvent

Любые важные операции записываются в Event Store.

117.3 Последствия

Плюсы:

- audit trail
 - аналитика
 - основа event-driven архитектуры
-

118 ADR-007 — Ticket Board architecture

Статус: Accepted

118.1 Контекст

Основной интерфейс системы — kanban board.

118.2 Решение

Создать endpoint:

GET /tickets/board

Board возвращает:

columns

cards

meta

118.3 Последствия

Плюсы:

- быстрый интерфейс
 - удобная работа с заявками
-

119 ADR-008 — Auto assignment

Статус: Accepted

119.1 Контекст

Система должна автоматически распределять заявки.

119.2 Решение

Алгоритм:

1. определить специализации категории
2. найти техников с этими специализациями
3. если autoAssignEnabled:

назначить первого кандидата

4. иначе:

оставить NEW

120 ADR-009 — Claim mechanism

Статус: Accepted

120.1 Контекст

Некоторые сервисные компании используют pull модель.

120.2 Решение

Техники могут забирать заявки.

API:

GET /tickets/available
POST /tickets/:id/claim

121 ADR-010 — Docker development environment

Статус: Accepted

121.1 Контекст

Необходимо одинаковое окружение разработки.

121.2 Решение

Использовать Docker.

Контейнеры:

backend
postgres

122 ADR-011 — Prisma ORM

Статус: Accepted

122.1 Контекст

Необходима удобная работа с PostgreSQL.

122.2 Решение

Использовать Prisma ORM.

Причины:

- типизация
 - миграции
 - удобный API
-

123 ADR-012 — Architecture documentation

Статус: Accepted

123.1 Контекст

Сложные системы требуют документации.

123.2 Решение

Поддерживать архитектурные документы:

PLATFORM_CONSTITUTION
ARCHITECTURE
SECURITY_MODEL
TICKET_VISIBILITY_MATRIX
AI_CONTEXT

124 Добавление новых ADR

Каждое новое решение добавляется как:

ADR-XXX

Структура записи:

Контекст

Решение

Последствия